FIAP ON

DJANGO WEB FRAMEWORK

IMAGENS, FILTROS E BUSCAS

ANDREY MASIERO

4

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 4.1 – Campo indicativo de empréstimo criado com sucesso.....6
Figura 4.2 – Permissão de edição do campo emprestado através de acesso rápido 7
Figura 4.3 – Resultado dos livros filtrados pela disponibilidade 9
Figura 4.4 – Resultado dos livros filtrados pela disponibilidade 11
Figura 4.5 – Exibição das capas dos livros 13

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 4.1 – Alterando ExibeLivro para permitir edição rápida 6
Código-fonte 4.2 – Inclusão da rota para empréstimo.....7
Código-fonte 4.3 – Alterando a reserva para empréstimo.....8
Código-fonte 4.4 – Filtrando apenas os livros disponíveis 8
Código-fonte 4.5 – Versão do modelo com permissão de upload de imagens 10
Código-fonte 4.6 – Inclusão do diretório de mídia.....11
Código-fonte 4.7 – Inclusão do diretório de mídia.....12
Código-fonte 4.8 – Validação da imagem da capa.....13
Código-fonte 4.9 – Inclusão da url para busca de livros.....14
Código-fonte 4.10 – Método busca incluso no arquivo views.py.....14
Código-fonte 4.11 – Partial do formulário de busca 15
Código-fonte 4.12 – Método busca completo. 15

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

4 IMAGENS, FILTROS E BUSCAS 5
4.1 Aprimorando o sistema 5
4.2 Alterando o modelo 5
4.3 Listando o que pode ser emprestado 7
4.4 Adicionando uma capa 9
4.5 Quem vê a capa... acha mais rápido 11
4.6 Em busca do livro querido 13
REFERÊNCIAS 17

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 IMAGENS, FILTROS E BUSCAS

4.1 Aprimorando o sistema

Nosso site está funcionando muito bem. Porém podemos melhorar algumas coisas nele, pois ainda não existem as imagens das capas de cada livro. Também não conseguimos dizer se um livro foi emprestado ou não. E a partir dessa informação, será que conseguimos realizar um filtro para exibir apenas as informações dos livros que gostaríamos de reservar? Sim, tudo isso é possível, e vamos implementar a busca pelo formulário também. Vamos iniciar pela parte do modelo, inserindo as imagens e a flag de empréstimo.

4.2 Alterando o modelo

Iniciamos alterando o modelo para que aceite a opção de empréstimo. Na classe Livro, adicione um novo campo chamado emprestado e atribua um campo booleano, utilizando o objeto models.BooleanField. O primeiro argumento é o valor default do item como False. Desta maneira, quando criamos um livro, ele já permanece pronto para empréstimo.

Como alteramos o modelo, devemos enviar a atualização para o banco de dados através do comando `python manage.py makemigrations` no terminal, para criar a migração. Feito isso, efetivamos a migração com migrate da mesma forma que fizemos anteriormente.

Após esse passo, se acessarmos a área de edição do livro no site de administrador, conseguiremos acessar esta opção nova de empréstimo do livro. Veja o resultado apresentado na Figura “Campo indicativo de empréstimo criado com sucesso”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

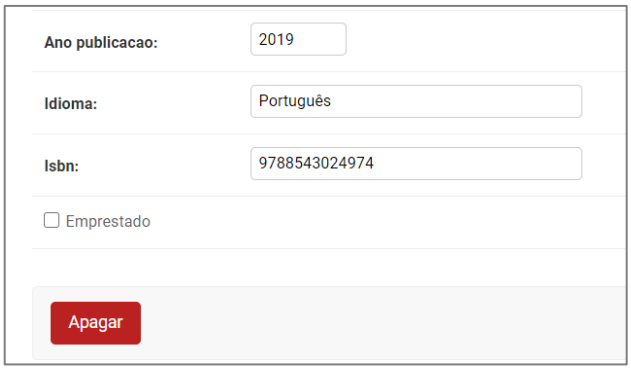

Figura 4.1 – Campo indicativo de empréstimo criado com sucesso
Fonte: Elaborado pelo autor (2020)

Como essa é uma informação que poderemos acessar para alterar com muita frequência, podemos facilitar o acesso à alteração dela diretamente pela tabela de listagem dos livros na parte de administração. Vamos alterar a classe ExibeLivro com as opções apresentadas no Código-fonte “Alterando ExibeLivro para permitir edição rápida”.

```
from django.contrib import admin
from .models import Livro


class ExibeLivro(admin.ModelAdmin):
    list_display = ('id', 'titulo', 'ano_publicacao', 'isbn',
'emprestado')
    list_display_links = ('id', 'titulo')
    search_fields = ('titulo',)
    list_filter = ('ano_publicacao',)
    list_editable = ('emprestado',)
    list_per_page = 2


admin.site.register(Livro, ExibeLivro)
```

Código-fonte 4.1 – Alterando ExibeLivro para permitir edição rápida
Fonte: Elaborado pelo autor (2020)

Adicionamos o campo emprestado para aparecer na tabela principal e em seguida informamos como um campo editável pela propriedade list_editable. Confira o resultado esperado conforme a Figura “Permissão de edição do campo emprestado através de acesso rápido”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 4.2 – Permissão de edição do campo emprestado através de acesso rápido
Fonte: Elaborado pelo autor (2020)

Maravilha, está funcionando, porém não é aqui que o cliente terá acesso a essa informação. Para isso, vamos construir uma rota de empréstimo e alterar o menu Reservar para Emprestar. Nele, apenas livros disponíveis para empréstimos serão exibidos.

4.3 Listando o que pode ser emprestado

O primeiro passo é criar uma url para empréstimo no arquivo urls.py. O Código-fonte “Inclusão da rota para empréstimo” apresenta a atualização do arquivo urls.py.

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('livro/<int:livro_id>', views.livro, name='livro'),
    path('emprestimo', views.emprestimo, name='emprestimo'),
]
```

Código-fonte 4.2 – Inclusão da rota para empréstimo
Fonte: Elaborado pelo autor (2020)

Na sequência, temos o Código-fonte “Alterando a reserva para empréstimo” com a alteração da partial cabecalho.html. Como criamos a partial, a alteração já será realizada para o sistema como um todo. Veja a seguir como ficou o código.

```
<header class="container">
    <a class="navbar-brand" href="{% url 'index' %}">
        <i class="fas fa-book-open"></i>
        Biblioteca
    </a>
    <button
            class="navbar-toggler"
            type="button"
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            data-toggle="collapse"
            data-target="#navbarNavAltMarkup"
            aria-controls="navbarNavAltMarkup"
            aria-expanded="false"
            aria-label="Toggle navigation"
    >
        <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse"
id="navbarNavAltMarkup">
        <div class="navbar-nav">
            <a class="nav-link" href="{% url 'index'
%}">Livros</a>
            <a class="nav-link" href="{% url 'emprestimo'
%}">Empréstimo</a>
        </div>
    </div>
</header>
```

Código-fonte 4.3 – Alterando a reserva para empréstimo
Fonte: Elaborado pelo autor (2020)

O próximo passo é ir ao arquivo views.py e criar o método empréstimo. Ele filtrará os livros retornando apenas os disponíveis para empréstimo. Confira o uso da função filter através do Código-fonte “Filtrando apenas os livros disponíveis”.

```
from django.shortcuts import render, get_object_or_404
from .models import Livro

def index(request):
    dados = {
        'livros': Livro.objects.all()
    }
    return render(request, 'index.html', dados)


def livro(request, livro_id):
    livro = {
        'livro': get_object_or_404(Livro, pk=livro_id)
    }
    return render(request, 'livro.html', livro)

def emprestimo(request):
    dados = {
        'livros': Livro.objects.filter(emprestado=False)
    }
    return render(request, 'index.html', dados)
```

Código-fonte 4.4 – Filtrando apenas os livros disponíveis
Fonte: Elaborado pelo autor (2020)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Como o arquivo de exibição terá o mesmo layout que a página index.html, utilizaremos a mesma página, porém informaremos os dados filtrados de acordo com a disponibilidade dos livros. O resultado deve ser o mesmo apresentado na Figura “Resultado dos livros filtrados pela disponibilidade”.

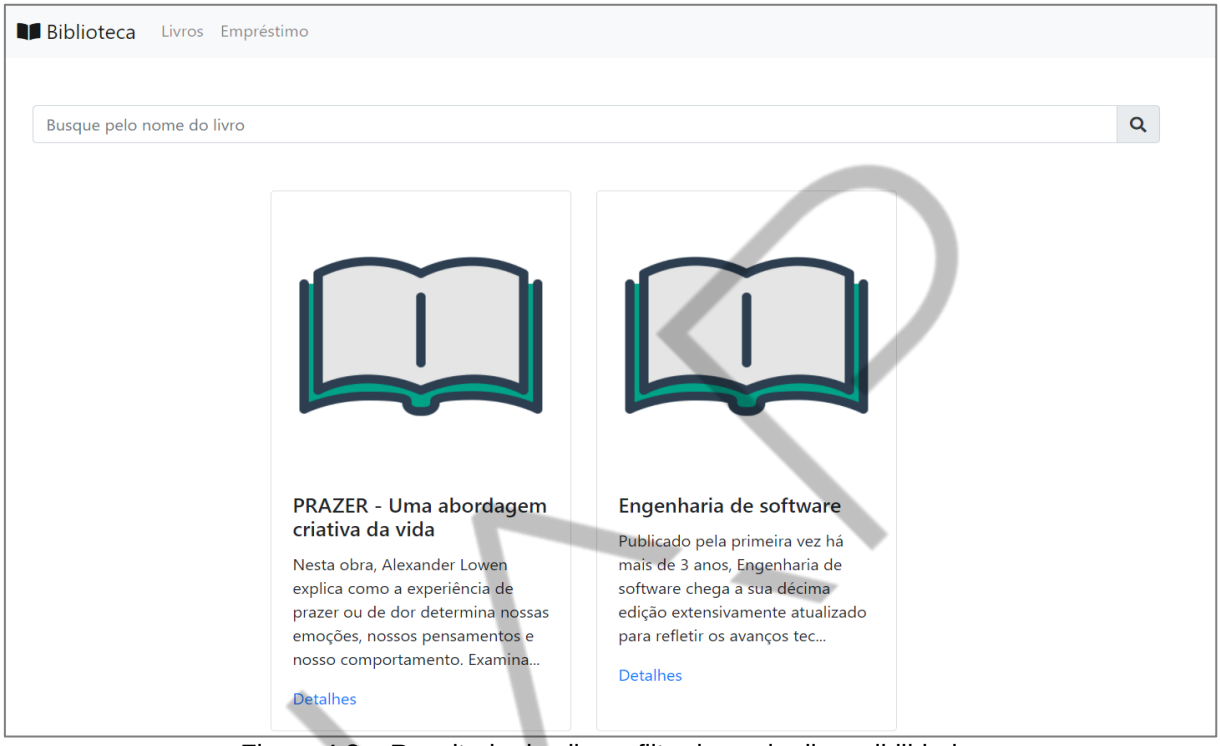

Figura 4.3 – Resultado dos livros filtrados pela disponibilidade
Fonte: Elaborado pelo autor (2020)

Vamos agora disponibilizar as capas dos livros para que a identificação seja mais tranquila.

4.4 Adicionando uma capa

Para adicionarmos a capa, faremos o mesmo processo realizado para a propriedade de empréstimo. Vamos começar incluindo um novo campo para armazenar a imagem. Chamaremos de capa e atribuiremos o objeto models.ImageField, que recebe o local ao qual a foto será enviada via upload. Os projetos até então são executados localmente. Portanto, precisamos apenas informar o diretório da máquina, onde serão descarregadas as fotos, o qual deve ser exclusivo para essa finalidade.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Dentro de models.ImageField, aplicamos a propriedade upload_to para indicar o caminho sendo igual a capas/, seguido de %d, %m e %Y para ser baseado no dia, mês e ano de criação e, assim, evitaremos problemas de sincronização conforme os livros são cadastrados. Utilizaremos uma segunda propriedade para permissão de valores em branco. Para isso, adicionamos blank=True permitindo deixar em branco — caso o item não possua uma imagem própria. O Código-fonte “Versão do modelo com permissão de upload de imagens” apresenta o resultado da alteração da classe modelo Livro.

```
from django.db import models
from autores.models import Autor


class Livro(models.Model):
    autor = models.ForeignKey(Autor,
on_delete=models.CASCADE)
    titulo = models.CharField(max_length=200)
    descricao = models.TextField()
    paginas = models.IntegerField()
    editora = models.CharField(max_length=50)
    edicao = models.IntegerField()
    ano_publicacao = models.IntegerField()
    idioma = models.CharField(max_length=20)
    isbn = models.CharField(max_length=20)
    capa = models.ImageField(upload_to='capas/%d/%m/%Y',
blank=True)
    emprestado = models.BooleanField(default=False)
```

Código-fonte 4.5 – Versão do modelo com permissão de upload de imagens
Fonte: Elaborado pelo autor (2020)

Na sequência, acessamos setting.py de "library" para criarmos dois comportamentos na aplicação. Eles indicam o uso de arquivos de mídia como imagens do site. Temos uma rota para arquivos estáticos e adicionamos outra rota para esses novos tipos de documentos, para possibilitar o upload das fotos.

Após a parte dos arquivos estáticos, insira MEDIA_ROOT sendo nosso próprio diretório os.path.join(), passando BASE_DIR, seguido de 'media'. Em seguida, escrevemos outra configuração chamada MEDIA_URL para referenciar '/media/', pois no banco de dados é salvo o caminho para uma determinada imagem, e não o próprio arquivo da imagem. O Código-fonte “Inclusão do diretório de mídia” apresenta o final do arquivo settings.py.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
STATIC_ROOT = os.path.join(BASE_DIR, 'static')
STATIC_URL = '/static/'
STATICFILES_DIRS = [
    os.path.join(BASE_DIR, 'library/static')
]

MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
MEDIA_URL = '/media/'
```

Código-fonte 4.6 – Inclusão do diretório de mídia
Fonte: Elaborado pelo autor (2020)

Nesse momento, o seu servidor pode gerar um problema, pois para trabalhar com imagens é necessário instalar o módulo Pillow. Para isso, no terminal digite o comando pip install pillow e em seguida fazemos o processo de migração para o banco de dados.

Caso tudo ocorra bem, no sistema administrador identificaremos um campo para fazer o upload da imagem. Conforme Figura “Resultado dos livros filtrados pela disponibilidade”.

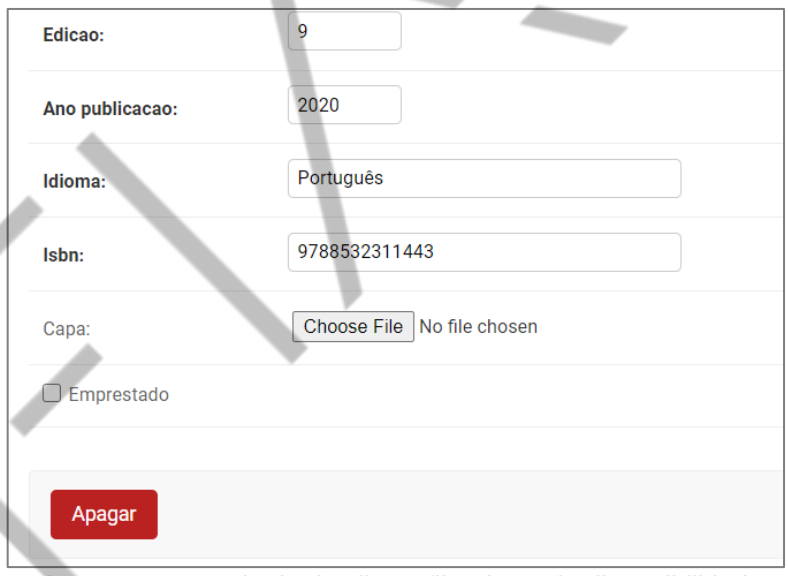

Figura 4.4 – Resultado dos livros filtrados pela disponibilidade
Fonte: Elaborado pelo autor (2020)

O próximo passo será exibir as capas na tela.

4.5 Quem vê a capa... acha mais rápido

Após salvarmos as capas de cada livro, precisamos apresentá-las nos Cards criados para exibi-los na página de detalhes. Sendo assim, vamos acessar urls.py em library para indicar o uso das configurações de mídia recém incluídas em settings.py.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Inclua um sinal de + ao final de urlpatterns, seguido de static() que recebe setting.MEDIA_URL, onde estão os caminhos, seguido de document_root igual a settings.MEDIA_ROOT. O Código-fonte “Inclusão do diretório de mídia” apresenta como deve ficar o arquivo urls.py do aplicativo principal nomeado de library.

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('', include('livros.urls')),
    path('admin/', admin.site.urls),
] + static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)
```

Código-fonte 4.7 – Inclusão do diretório de mídia
Fonte: Elaborado pelo autor (2020)

Desta forma, dizemos que os arquivos salvos via upload podem ter suas urls utilizadas pela aplicação. Agora, criamos uma verificação que avalia se existe uma capa; caso não exista, carregaremos a imagem padrão que definimos. Para isso, basta acessar o index.html e inserimos o Código-fonte “Validação da imagem da capa” para validar a parte da capa.

```
        {% for livro in livros %}
        <div class="card mr-4 mb-3" style="width: 18rem">
            {% if livro.capa == '' %}
                <img src="{% static 'img/book.png'%}"
class="card-img-top"/>
            {% else %}
                <img src="{{ livro.capa.url }}" class="card-
img-top"/>
            {% endif %}
            <div class="card-body">
                <h5 class="card-title">{{ livro.titulo
}}</h5>
                <p class="card-text">
                    {{ livro.descricao|slice:':150'|add:'...'
}}
                </p>
                <a href="{% url 'livro' livro.id %}"
class="card-link">Detalhes</a>
            </div>
        </div>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        {% endfor %}
```

Código-fonte 4.8 – Validação da imagem da capa
Fonte: Elaborado pelo autor (2020)

Dentro do for de livros, escrevemos código para processamento {% %} recebendo if para verificar se o livro.capa é igual a nulo, sendo verdade, exibe a imagem padrão. Caso contrário, o site deve apresentar a url dos arquivos de mídia configurada na aplicação, de maneira automática e disponível através da propriedade livro.capa.url. O resultado é apresentado na Figura “Exibição das capas dos livros”.

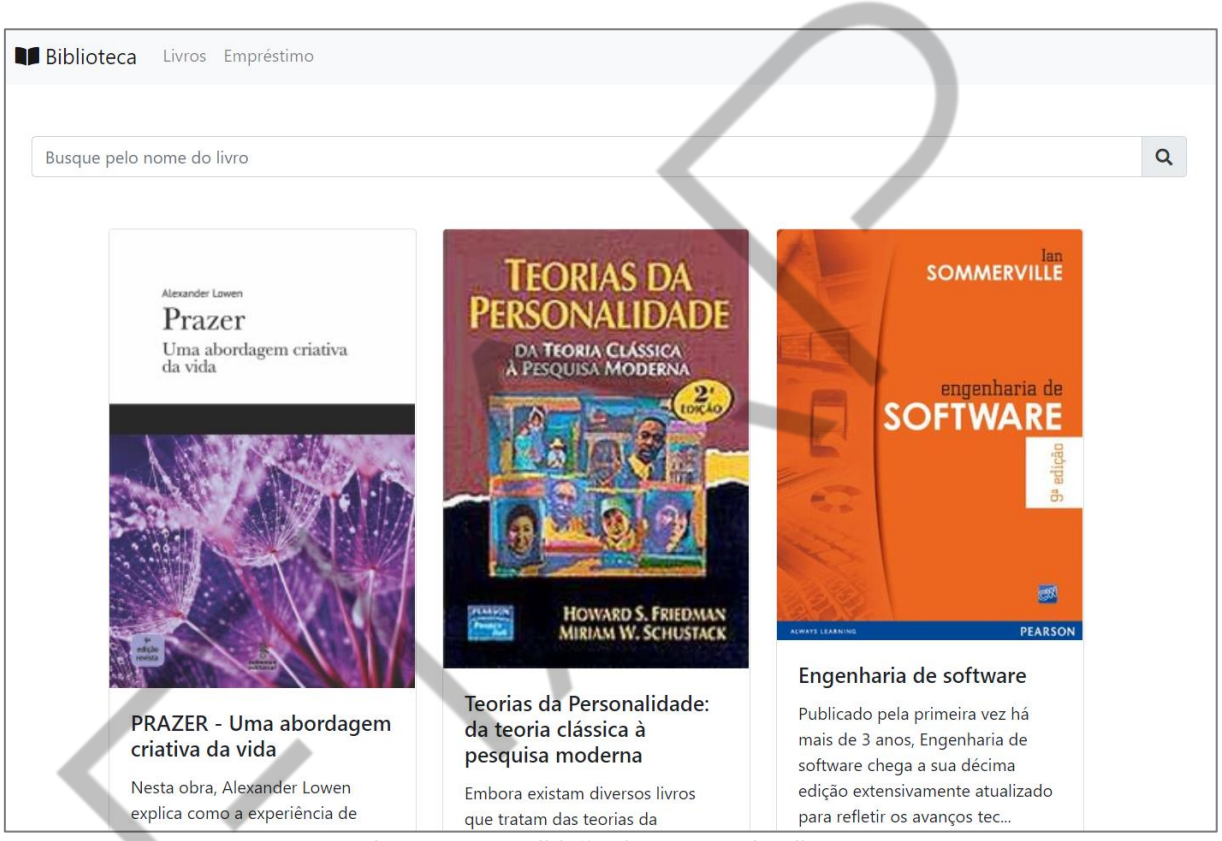

Figura 4.5 – Exibição das capas dos livros
Fonte: Elaborado pelo autor (2020)

Agora, basta fazer a mesma coisa para a página de detalhe do livro. O próximo passo que temos de fazer em nosso sistema é deixar o formulário de busca funcionando. E é exatamente o que faremos.

4.6 Em busca do livro querido

Para fazermos a busca funcionar corretamente, criaremos a url responsável por acionar esse recurso e a chamaremos de busca, conforme apresentado no Código-fonte “Inclusão da url para busca de livros”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
urlpatterns = [
    path('', views.index, name='index'),
    path('livro/<int:livro_id>', views.livro, name='livro'),
    path('emprestimo', views.emprestimo, name='emprestimo'),
    path('busca', views.busca, name='busca'),
]
```

Código-fonte 4.9 – Inclusão da url para busca de livros
Fonte: Elaborado pelo autor (2020)

Na sequência, vamos criar o método de busca dentro do arquivo views.py. Ele também renderizará a página index, a princípio. O Código-fonte “Método busca incluso no arquivo views.py” apresenta a inclusão do método busca no arquivo views.py e a inclusão de ordenação pelo título do livro no método da index.

```
from django.shortcuts import render, get_object_or_404
from .models import Livro


def index(request):
    dados = {
        'livros': Livro.objects.order_by('titulo').all()
    }
    return render(request, 'index.html', dados)


def livro(request, livro_id):
    livro = {
        'livro': get_object_or_404(Livro, pk=livro_id)
    }
    return render(request, 'livro.html', livro)


def emprestimo(request):
    dados = {
        'livros': Livro.objects.filter(emprestado=False)
    }
    print(dados)
    return render(request, 'index.html', dados)


def busca(request):
    return render(request, 'index.html')
```

Código-fonte 4.10 – Método busca incluso no arquivo views.py
Fonte: Elaborado pelo autor (2020)

Vamos alterar um pouco a parte do formulário. Como sabemos — esse é um código que precisa ser replicado em outras páginas — também já criaremos uma partial para ele. Uma boa prática para os nomes de partials é utilizar o underline no

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

início do nome. Nesta, vamos chamar de _buscar.html e incluiremos o conteúdo de acordo com o Código-fonte “Partial do formulário de busca”.

```
<form class="form-inline" action="{% url 'busca' %}">
    <div class="input-group mt-5 mb-5 col-12">
        <input
                name="titulo"
                type="text"
                class="form-control"
                placeholder="Busque pelo nome do livro"
        />
        <div class="input-group-append">
            <div class="input-group-text">
                <a class="text-dark" href="#"
onclick="document.querySelector('form').submit()">
                    <i class="fas fa-search"></i>
                </a>
            </div>
        </div>
    </div>
</form>
```

Código-fonte 4.11 – Partial do formulário de busca
Fonte: Elaborado pelo autor (2020)

Agora, vamos trabalhar no código do método busca. Faça de acordo com o Código-fonte “Método busca completo”.

```
def busca(request):
    livros = Livro.objects.order_by('titulo').all()

    if 'titulo' in request.GET:
        titulo = request.GET['titulo']
        if titulo:
            livros = livros.filter(titulo__icontains=titulo)

    dados = {
        'livros': livros
    }

    return render(request, 'index.html', dados)
```

Código-fonte 4.12 – Método busca completo.
Fonte: Elaborado pelo autor.(2020)

O primeiro passo é recuperarmos todos os livros do banco de dados. Na sequência, verifica-se se existe, dentro no método GET da requisição, um parâmetro chamado titulo. Em caso positivo, recuperamos as informações disponíveis do parâmetro utilizando um acesso de dicionário. Se esse parâmetro tiver conteúdo, efetuamos um filtro com a opção __icontains, onde ela verifica se a substring existe

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

em qualquer parte do campo procurado. Por fim, o filtro de livros aplicado é retornado à página index.html através do método render.

A partir deste momento, precisamos criar acesso ao sistema e permitir que o usuário autenticado realize o empréstimo do livro e o devolva. Mas isso é um assunto para o próximo capítulo.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

Django. **Documentação do Django Web Framework**. [s.d.]. Disponível em: https://www.djangoproject.com/. Acesso em: 05 out. 2020.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com